

3.7 Debugging Code in Python

ڈیباگنگ یعنی غلطی ڈھونڈ کر ٹھیک کرنا۔

CS XI

Chapter 3: Programming Fundamentals

Lecture 13 of 13

3.7 Debugging Code in Python

LEARN IT

Debugging is the process of identifying, tracing and fixing bugs so a program works correctly. Every program you write will have bugs. The skill is not "never make a mistake"; it is "find the mistake quickly".

Python errors fall into three families:

Type	When it appears	What you notice	Example
Syntax error	Before the program even starts	Python refuses to run	<code>print("Hello" — missing)</code>
Runtime error (exception)	While the program is running	It crashes with a traceback	<code>x = 10 / 0 → ZeroDivisionError</code>
Logical error	Program finishes "successfully"	The output is wrong	<code>area = length + width</code> instead of <code>*</code>

WORKED EXAMPLE

Reading a traceback. When Python crashes it prints a roadmap: the file, the line number, and the error type.

```
x = int("abc")
```

OUTPUT

```
Traceback (most recent call last):
  File "debug_program.py", line 1, in <module>
    x = int("abc")
ValueError: invalid literal for int() with base 10: 'abc'
```

Diagnosis: on **line 1** we tried to convert the letters "abc" into an integer, which raised a `ValueError`.

EXAM TIP

Examiners love "identify the error type" questions. Ask: (1) Does it refuse to start? → syntax. (2) Does it crash mid-run? → runtime. (3) Does it run but give a wrong answer? → logical. Write the name of the error **and** how to fix it.

REMEMBER IT

Fix syntax errors first (the program cannot start). Then run it. Then check the output against a hand-worked example to catch logical errors.

COMMON MISTAKE

A logical error is the most dangerous because Python reports nothing. Always test with values whose answer you already know (for a rectangle 3 by 4, area must be 12, not 7).

اردو میں سمجھیں

ڈیبگنگ یعنی غلطی ڈھونڈ کر ٹھیک کرنا۔ نحوی غلطی پر پروگرام چلتا ہی نہیں، رن ٹائم پر چلتے چلتے رک جاتا ہے، منطقی غلطی پر چل تو جاتا ہے مگر جواب غلط دیتا ہے — یہ سب سے خطرناک ہے۔

CHECK YOURSELF

1. `print("Hi"` — which error type?
2. A program that should add two numbers multiplies them instead. Which error type?
3. `int("12.5")` raises which named exception?

Answers: 1. Syntax error. 2. Logical error. 3. `ValueError` (runtime).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. A syntax error is:

- a. Broken grammar; the program will not run
- b. Wrong answer but it runs
- c. A crash while running
- d. A slow Big O

MCQ 2. A runtime error is:

- a. A crash while running (e.g. /0, missing file)
- b. A missing colon only
- c. A wrong formula that still prints
- d. A comment

MCQ 3. A logic error is:

- a. Runs but the answer is wrong
- b. Always a crash
- c. Always SyntaxError
- d. A font

MCQ 4. A traceback tells you:

- a. File, line, error type, message
- b. OSI layer only
- c. The teacher's marks
- d. Big O

MCQ 5. `int("hi")` raises:

- a. `ValueError` (runtime)
- b. A logic silent bug
- c. `SyntaxError` always
- d. Success

MCQ 6. Using `=` instead of `==` in if is usually:

- a. `SyntaxError`
- b. A correct if
- c. A sort
- d. A pie

Extra short questions

1. Name the three error types with one example each.
2. How do you read a traceback from the bottom?
3. How do you hunt a logic error?
4. Does a logic error show a traceback?

Extra long question

1. Classify five bugs: missing colon; `int(input)` on "A"; average without divide; wrong indent of else; open missing file. For each give type + fix.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Syntax: if `x==1` print (missing `:`). Runtime: `1/0`. Logic: average = `a+b` without `/2`.
2. Last line is the error; above it the call stack; look at your file's line number.
3. Print/trace variables, test edge cases, compare with a worked example.
4. No — that is why they are dangerous.

Long

1. Syntax; runtime `ValueError`; logic; syntax/logic indent; runtime `FileNotFoundError`. Fixes: add colon; validate input; `/n`; match indent; check path/try-except.